Towards a Machine Learning-Assisted Kernel with LAKE

Authors: Henrique Fingler, Isha Tarte, Hangchen Yu, Ariel Szekely, Bodun Hu, Aditya Akella, Christopher J. Rossbach

Presented By: Kaushik Kurapati

Why Kernel Decision-Making Is Getting Harder

- Modern operating systems manage increasingly complex hardware.
- Examples include multicore CPUs, NUMA, new memory technologies, and accelerators.
- Kernel subsystems such as memory management, scheduling, and file systems still rely heavily on hand-tuned heuristics.
- These heuristics aim for reasonable average-case behavior, but they are hard to keep optimal across diverse workloads and machines.

Why fixed heuristics are not enough

- Kernel heuristics are typically a one-size-fits-all solution.
- Different servers and workloads often use the same policies even when their behavior is very different.
- As system complexity grows, designing efficient general heuristics becomes harder.
- ML is attractive because it can learn workload-specific behavior instead of relying only on fixed rules.

Why Consider ML for Kernels?

- Prior work has used ML for:
  - CPU load balancing
  - File system prefetching
  - I/O latency prediction
  - Power and clock control
- The key idea is that ML may better navigate kernel tradeoff spaces than static heuristics.
- But most prior work focused on a single subsystem, not the broader systems integration problem.

The Real Problem the Paper Tackles

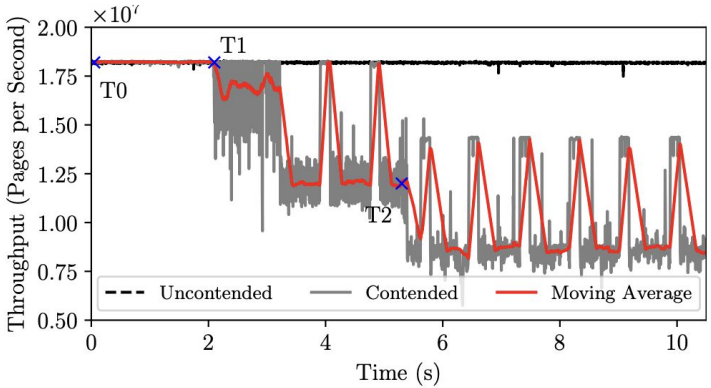
- The paper is less about inventing one new ML policy.
- It is mainly about the systems infrastructure needed to make ML-assisted kernels practical.
- The authors argue that common infrastructure is needed to avoid ad hoc, subsystem-specific implementations.
- They also argue that accelerators such as GPUs are often necessary because CPUs alone may not meet performance requirements.

Three Main Challenges

- C1: Accelerator access
  - GPUs and similar devices are hard to access from kernel space.
- C2: Variable profitability
  - GPU offload is not always worth it; benefits depend on workload, hardware, and batch size.
- C3: Feature collection
  - Useful features are spread across kernel layers and modules, often with asynchronous timing and locking constraints.

Why Naive GPU Sharing Fails

- Kernel ML work can contend with user-space applications for GPU resources.
- This is especially problematic because user applications often need stable accelerator performance.
- The paper shows that unmitigated contention can reduce user-space throughput by up to 68%.
- So the kernel cannot simply “always use the GPU.”



LAKE at a High Level

- LAKE = Learning-assisted, Accelerated KErnel
- LAKE is a framework that:
  - Exposes accelerator functionality to kernel subsystems
  - Manages when offload is worthwhile
  - Mitigates user/kernel contention,
  - Simplifies feature collection for ML inference.
- Main claim: learned kernels need reusable infrastructure, not just better models.

LAKE Architecture

- lakeLib
  - kernel-side API stubs
- lakeD
  - user-space daemon that executes remoted APIs
- lakeShm
  - Shared-memory path for efficient bulk data transfer
- Vendor libraries remain in user space, while kernel subsystems invoke them through LAKE.

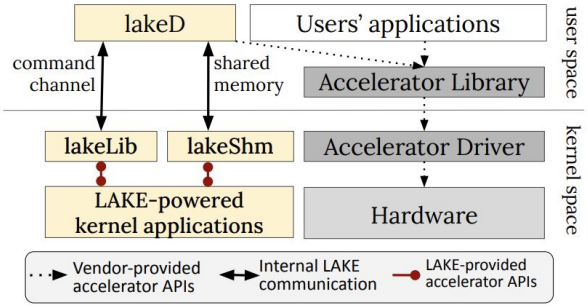


Figure 2: Overview of the architecture of LAKE.

API remoting workflow

- Kernel code calls an API exposed by lakeLib
- lakeLib serializes the call and sends it to lakeD
- lakeD invokes the real vendor library, such as CUDA
- Results are returned to kernel space
- lakeShm reduces copying overhead for bulk data movement

CPU vs GPU is a runtime policy decision

- Accelerators are only worthwhile past a certain batch size
- The paper calls this threshold the crossover point
- LAKE supports switching between CPU and GPU at function-call granularity
- Policies can use eBPF-style callbacks and runtime information
- The same policy framework is used for profitability and contention control

```
1  policy cu_policy(offload_func_t dev_func, void
       *dev_args, offload_func_t cpu_func, void *
       cpu_args) {
2    static nvmlUtilization_t util;
3    if ...5 ms elapsed since last check...
4      // LAKE-remoted nvml API
5      nvmlGetUtilization(dev, &util)
6    // compute GPU utilization moving average
7    int exec_rate = mov_avg(util.gpu);
8    // batch size for profitability threshold
9    int batch_sz = get_batch_size(def_args)
10   if (exec_rate < exec_threshold
11         && batch_sz >= batch_threshold)
12     return dev_func(dev_args);
13   else
14     return cpu_func(dev_args); }
```

Why LAKE needs an in-kernel feature registry

- Synchronous feature collection is often impractical
- Relevant features may be updated in different modules, at different times, on different threads
- LAKE provides APIs to:
  - Manage models
  - Capture features asynchronously
  - Build feature vectors
  - Retrieve batches
  - Run inference

Example: I/O latency prediction

- Features include pending I/Os and recent completion latencies
- Those values are observed at different code points:
  - I/O issue
  - I/O completion
- LAKE supports incremental, asynchronous feature construction across threads
- This makes kernel ML integration much easier than manual state management

```
// called when issuing a block I/O in Linux
generic_make_request_checks(struct bio *bio)
{
  sys = "bio_latency_prediction"
  // store this I/O's start time
  getnstimeofday(&(bio->io_start_ts));
  // increment pending I/Os on this device
  capture_feature_incr(dev,sys,"pend_ios",1)
  // this I/O becomes a feature vector
  commit_feature_capture(dev, sys, now())
  if(quantum passed or batch > thresh) {
    // get all features vectors in the ring
    fvs = get_features(dev, sys, NULL)
    // do inference on all feature vecs
    scores = score_features(dev, sys, fvs);
    // reject, re-issue or accept I/Os
    ...take action based on scores...
    // reset the feature vector ring
    truncate_features(dev, sys, NULL)
  }
  //start new feature
  begin_fv_capture(dev, sys, now())
  ...
```

```
// function called to end a block I/O
void bio_endio(struct bio *bio) {
  sys = "bio_latency_prediction"
  // get latency of this I/O
  lat = get_io_latency(bio->io_start_ts);
  // store this I/O's latency
  capture_feature(dev,sys,io_latencies,lat);
  // one less pending I/O on this device
  capture_feature_incr(dev,sys,pend_ios,-1)
  ...
```

Implementation

- Prototype built on Linux 6.0
- LAKE exposes CUDA driver API 11.0, TensorFlow 2.4.0, and Keras 2.2.5 to kernel space
- Netlink is used for command/control because it has good latency
- lakeShm uses zero-copy shared memory for large transfers

Evaluation

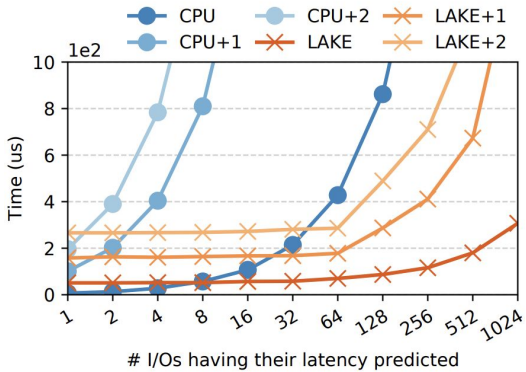
- I/O latency prediction
- Page warmth classification
- Load balancing
- File-system prefetching
- Malware detection
- File-system encryption

Important crossover points:

- I/O latency prediction: 8
- Page warmth: 1
- Load balancing: 128
- File-system prefetching: 64

End-to-end I/O latency case study

- LAKE preserves the benefits of ML-backed I/O latency prediction
- GPU inference becomes profitable for the original model at batch sizes greater than 8
- With larger models, the profitable batch size drops even further
- Example from the paper: for batch size 8, GPU inference can cut total time from about 120 µs on CPU to 86 µs, around a 28% reduction



Results

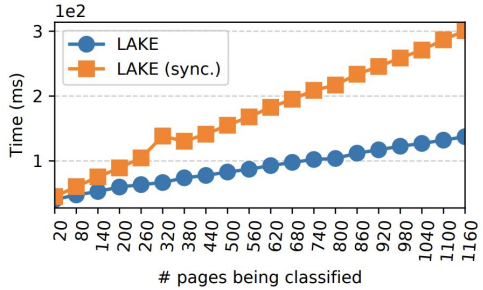


Figure 9: Time taken to predict page warmth through Kleio, a LSTM-based model for variable batch sizes. Data is shown only for LAKE (sync.) because data movement is handled synchronously by TensorFlow.

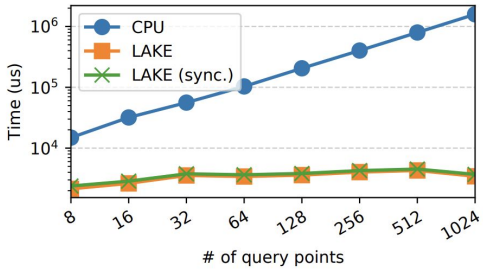


Figure 12: Time taken to predict if a sequence of syscalls could be from malicious software for various input sizes (number of syscalls in feature vector) using 4096 K-Nearest Neighbors (KNN) on a database of 16,384 reference points.

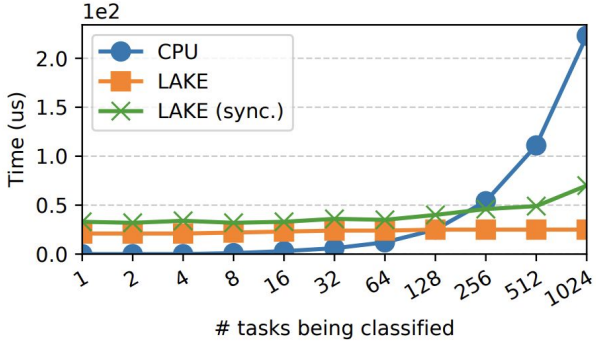


Figure 10: Time taken to predict load balancing decisions using MLLB in variable batch sizes.

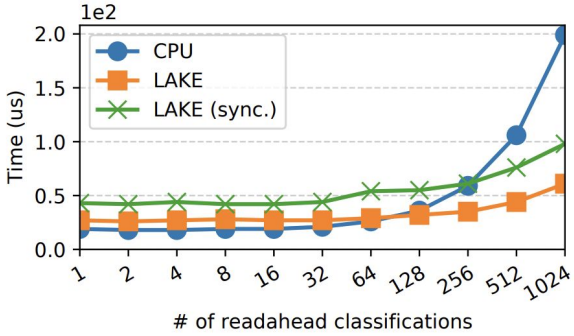


Figure 11: Time to predict file system readahead in variable batch sizes.

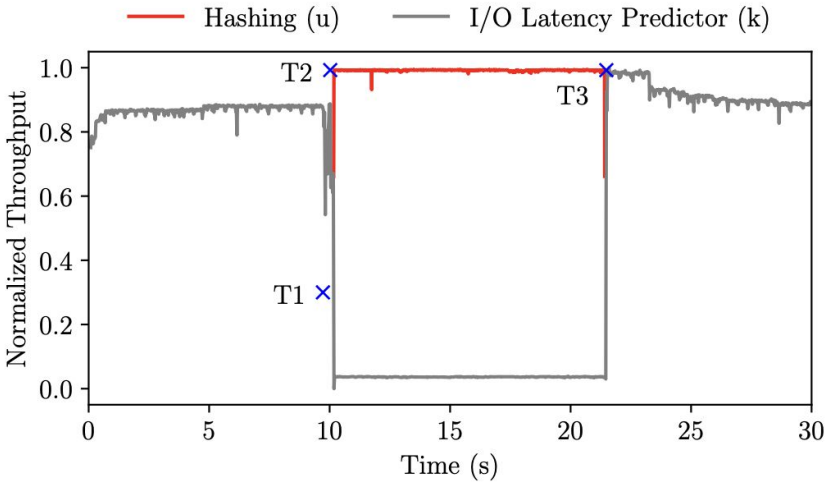


Figure 13: Kernel and user space throughput, normalized against peak throughput, under our adaptive contention-averse policy. At $T0$, our GPU-accelerated I/O latency classifier is running. At $T1$, a user space process that calculates hashes is launched. At $T2$, the user space process starts doing hashed on the GPU. LAKE detects contention for GPU compute resources and falls-back to CPU. At $T3$, the user space process terminates. LAKE detects that the GPU is not contended and switches execution back to the GPU.

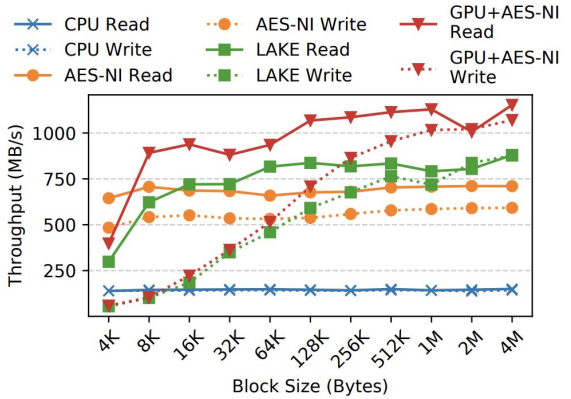


Figure 14: I/O throughput of AES-GCM-based eCryptfs, encrypting/decrypting on CPU, AES-NI, and a GPU through LAKE.

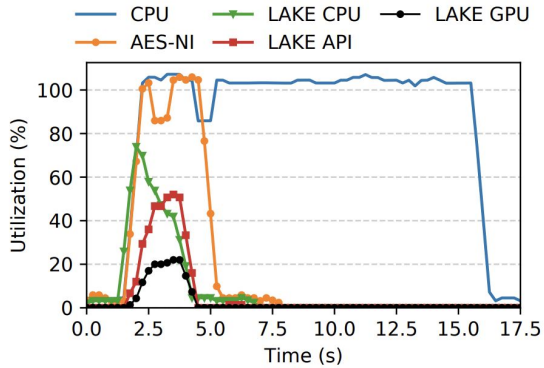


Figure 15: CPU and GPU utilization of reading a 2 GB file sequentially on eCryptfs with a block size of 2 MB, using CPU only, AES-NI and LAKE.

Strengths & Weaknesses

Strengths

- Strong systems contribution: focuses on infrastructure, not just model accuracy
- Realistic about tradeoffs: GPU is not always better
- Generality: evaluated across multiple workloads

Weaknesses

- Depends on a trusted user-space daemon
- Security and side-channel concerns are not fully solved
- Prototype is centered on Linux plus CUDA/NVIDIA
- Benefits are still workload- and hardware-dependent

Future Work

- Extend beyond CUDA/NVIDIA
- Improve daemon security and isolation
- Add policies for deciding whether ML should run at all
- Explore stronger online adaptation